

Python ile Uygulama, Veri ve Otomasyon

Python dilini yalnızca sözdizimi düzeyinde değil; ortam yönetimi, dosya/veri işleme, fonksiyonel araçlar, tip ipuçları, test, API kullanımı ve otomasyon akışlarıyla birlikte öğreten sentetik ders materyali.

Seviye

Başlangıç -> Orta

Önerilen süre

18-24 saat

Belge yapısı

15 sayfa / ders modülleri + uygulama

Kapsam: Python | Otomasyon | Veri işleme | Test

Amaç: Tek başına çalışılabilir, örnek ve mini uygulamalar içeren ders dokümanı.

MODÜL 0

Ders Haritası ve Hedefler

Bu doküman Python ile okunabilir, test edilebilir ve tekrar kullanılabilir küçük uygulamalar geliştirmeye odaklanır. Amaç yalnızca notebook üzerinde kod yazmak değil, gerçek proje düzeni ve otomasyon alışkanlığı kazandırmaktır.

Sayfa	Tema	Çıktı
3-5	Dil temelleri	Fonksiyon ve veri yapıları
6-10	OOP + Pythonic araçlar	İteratör, generator, decorator
11-12	Kalite + dış sistemler	Typing, test, HTTP
13-14	Veri + otomasyon	Pandas, CLI, packaging
15	Bitirme projesi	Raporlama otomasyonu

- venv ve pip ile izole proje ortamı kurar.
- List/dict/set ve comprehension yapılarını doğru seçer.
- Generator, decorator ve context manager mantığını açıklar.
- pytest ve type hints ile kalite sinyalleri oluşturur.
- CSV/JSON/API kaynaklarını birleştirerek otomatik rapor üretir.

Yaklaşım

Örnekler Python 3.11+ sözdizimi düşünülerek hazırlanmıştır; çoğu örnek daha eski 3.x sürümlerinde de küçük değişikliklerle çalışır.

MODÜL 1

Python Çalışma Ortamı ve Proje Kurulumu

Python yorumlanan bir dil olarak tanımlansa da CPython kaynak kodu bytecode biçimine dönüştürür ve sanal makine üzerinde yürütür. Proje bazlı virtual environment kullanmak bağımlılık çakışmalarını önler.

ORTAM KURULUMU

```
python -m venv .venv
# Windows
.venv\Scripts\activate
# Linux/macOS
source .venv/bin/activate

python -m pip install -U pip
pip install requests pytest pandas
```

Dosya / klasör	Amaç
pyproject.toml	Paket ve araç konfigürasyonu
src/	Uygulama kaynak kodu
tests/	Testler
.venv/	Yerel sanal ortam; repoya eklenmez

Mini görev

Yeni bir klasörde venv oluşturun, requests kurun ve `python -c "import requests; print(requests.__version__)"` komutuyla doğrulayın.

MODÜL 2

Temel Tipler ve Koleksiyonlar

Python dinamik tipli ancak güçlü tipli bir dildir. Bir değişken farklı zamanda farklı tipe işaret edebilir; fakat uyumsuz tipler otomatik olarak rastgele dönüştürülmez. list, tuple, dict ve set doğru veri modeli için temel araçlardır.

KOLEKSİYON ÖRNEĞİ

```
user = {
    "name": "Ada",
    "skills": ["Python", "SQL"],
    "active": True,
}

unique_skills = set(user["skills"])
name, *rest = ["Ada", "Linus", "Grace"]
```

Tip	Özellik
list	Sıralı, değiştirilebilir
tuple	Sıralı, immutable
dict	Anahtar-değer eşlemesi
set	Tekil elemanlar, hızlı üyelik testi

Pythonic seçim

Bir öğenin varlığını çok sık kontrol edecekseniz list yerine set daha uygun olabilir; veri sırasını korumak gerekiyorsa list tercih edin.

MODÜL 3

Kontrol Akışı, Fonksiyonlar ve Scope

Python girintiyi blok yapısının parçası olarak kullanır. Fonksiyonlar first-class object olduğundan değişkenlere atanabilir, başka fonksiyonlara verilebilir ve döndürülebilir. Keyword-only parametreler API okunabilirliğini artırır.

FONKSİYON + KONTROL

```
def calculate_total(items: list[float], *, tax: float = 0.20) -> float:
    subtotal = sum(items)
    return subtotal * (1 + tax)

for value in [10, 20, 30]:
    if value >= 20:
        print(value)

result = calculate_total([10, 15], tax=0.18)
```

- local scope fonksiyon içindeki isimleri kapsar.
- global kullanımı çoğu uygulamada sınırlı tutulmalıdır.
- *args pozisyonel, **kwargs isimli değişken sayıda argüman toplar.
- return yoksa fonksiyon None döndürür.

Mini görev

Bir not listesi alan ve min, max, ortalama değerlerini dict olarak döndüren summarize_scores fonksiyonu yazın.

MODÜL 4

Sınıflar, dataclass ve Composition

Python OOP yaklaşımı esnektir. Küçük veri taşıyıcılarında dataclass tekrar eden `__init__`/`__repr__` kodunu azaltır. Composition çoğu durumda derin kalıtım ağaçlarından daha kolay test edilir ve değiştirilir.

DATACLASS + COMPOSITION

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Product:
    id: int
    name: str
    price: float

class Basket:
    def __init__(self) -> None:
        self._items: list[Product] = []

    def add(self, product: Product) -> None:
        self._items.append(product)
```

Araç	Ne zaman?
@dataclass	Yapısal veri modeli
property	Kontrollü attribute erişimi
classmethod	Sınıfa bağlı alternatif constructor
Protocol	Duck typing için statik sözleşme

Tasarım notu

Python private alanları dil seviyesinde tamamen gizlemez; `_name` biçimi geliştiriciler arası "iç kullanım" sözleşmesidir.

MODÜL 5

Dosyalar, pathlib, CSV ve JSON

Otomasyon işlerinin önemli bölümü dosyaları bulma, okuma, dönüştürme ve kaydetme adımlarından oluşur. pathlib işletim sistemi bağımsız yol yönetimini kolaylaştırır. with bloğu dosyanın her koşulda kapanmasını sağlar.

PATHLIB + JSON

```
from pathlib import Path
import json

config_path = Path("config.json")
config = json.loads(config_path.read_text(encoding="utf-8"))

out = Path("output")
out.mkdir(exist_ok=True)
(out / "result.json").write_text(
    json.dumps(config, ensure_ascii=False, indent=2),
    encoding="utf-8",
)
```

- csv.DictReader satırları dict olarak okumayı kolaylaştırır.
- encoding="utf-8" açıkça belirtmek platform farklarını azaltır.
- Path.glob / rglob dosya desenleriyle arama yapar.
- Büyük dosyalarda tüm içeriği tek seferde belleğe almak yerine satır satır işleme düşünülebilir.

Mini görev

Bir klasördeki tüm .csv dosyalarını rglob ile bulun ve dosya adlarını, boyutlarını ve satır sayılarını report.json içine yazın.

MODÜL 6

Exception, Logging ve Sağlam Kod

Exception mekanizması beklenmeyen durumları yönetir. except bloğu mümkün olduğunca spesifik exception tipini yakalamalıdır. logging modülü print yerine seviye, zaman ve bağlam içeren gözlemlenebilir çıktı sağlar.

HATA YÖNETİMİ

```
import logging

log = logging.getLogger(__name__)

def load_number(text: str) -> int | None:
    try:
        return int(text)
    except ValueError:
        log.warning("Invalid integer: %r", text)
        return None
```

Seviye	Örnek kullanım
DEBUG	Ayrıntılı teşhis
INFO	Normal iş akışı olayı
WARNING	Beklenmeyen ama yönetilebilir durum
ERROR	İşlem başarısız, müdahale gerekebilir

Kaçınılacak hata

except Exception: pass hatayı sessizce yok eder. En azından loglamak ve mümkünse daha dar exception türü kullanmak gerekir.

MODÜL 7

Comprehension, Iterator ve Generator

Comprehension yapıları basit dönüşüm ve filtrelemeyi kısa ifade eder. Generator ise tüm sonucu belleğe almak yerine değerleri ihtiyaç oldukça üretir. Bu, büyük veri akışlarında bellek kullanımını ciddi biçimde azaltabilir.

COMPREHENSION + GENERATOR

```
squares = [x * x for x in range(10) if x % 2 == 0]

def read_clean_lines(path):
    with open(path, encoding="utf-8") as f:
        for line in f:
            text = line.strip()
            if text:
                yield text

for line in read_clean_lines("big.txt"):
    process(line)
```

- iter(obj) iterator üretir; next(iterator) sıradaki değeri ister.
- yield kullanan fonksiyon generator döndürür.
- Generator expression: (x*x for x in values).
- Tek kullanımlık iterator yeniden başlatılamayabilir; tekrar gerekirse yeni iterator üretin.

Performans fikri

Milyonlarca satırlık log dosyasında list comprehension ile tüm satırları belleğe almak yerine generator pipeline kullanın.

MODÜL 8

Decorator ve Context Manager

Decorator bir fonksiyon veya sınıfın davranışını sarmalayarak ek işlev kazandırır. Context manager ise kaynak edinme ve bırakma adımlarını with ifadesiyle güvenli hale getirir. Zaman ölçme, retry, yetki kontrolü ve transaction sınırları tipik örneklerdir.

DECORATOR + CONTEXT MANAGER

```
from contextlib import contextmanager
from time import perf_counter

def timed(fn):
    def wrapper(*args, **kwargs):
        start = perf_counter()
        try:
            return fn(*args, **kwargs)
        finally:
            print(fn.__name__, perf_counter() - start)
    return wrapper

@contextmanager
def managed_resource(resource):
    try: yield resource
    finally: resource.close()
```

İyileştirme

Gerçek decoratorlarda functools.wraps kullanarak sarılan fonksiyonun ad ve dokümantasyon meta verilerini koruyun.

- @decorator sözdizimi, fonksiyonu decorator(function) biçiminde yeniden bağlar.
- __enter__/__exit__ özel metotları sınıf tabanlı context manager oluşturur.
- contextlib.contextmanager generator tabanlı kısa alternatif sağlar.

MODÜL 9

Type Hints, pytest ve Kod Kalitesi

Python type hints çalışma zamanında zorunlu değildir; ancak IDE, static checker ve ekip iletişimi için güçlü bir sözleşme sağlar. pytest küçük ve okunabilir test fonksiyonlarıyla davranışı güvenceye alır.

TYPING + TEST

```
from typing import Protocol

class PriceSource(Protocol):
    def get_price(self, code: str) -> float: ...

def discounted(source: PriceSource, code: str, rate: float) -> float:
    return source.get_price(code) * (1 - rate)

def test_discounted(fake_source):
    assert discounted(fake_source, "A", 0.10) == 90.0
```

Araç	Amaç
ruff	Lint ve hızlı stil kontrolleri
black	Otomatik formatlama
mypy/pyright	Statik tip kontrolü
pytest	Test çalıştırma ve fixture altyapısı

Test ilkesi

Deterministic test yazın: gerçek zaman, rastgelelik ve ağ bağımlılıklarını kontrol altına alın veya test double kullanın.

MODÜL 10

HTTP API Kullanımı ve Veri Doğrulama

Birçok Python otomasyonu REST API ile veri alır veya gönderir. requests basit istemci kullanımını kolaylaştırır. Timeout belirtmek, status code kontrol etmek ve beklenmeyen şemaları doğrulamak üretim kalitesi için kritiktir.

HTTP İSTEMCİSİ

```
import requests

def fetch_users(base_url: str) -> list[dict]:
    response = requests.get(f"{base_url}/users", timeout=10)
    response.raise_for_status()
    data = response.json()
    if not isinstance(data, list):
        raise ValueError("Expected a list")
    return data
```

- timeout olmadan istek teorik olarak uzun süre bekleyebilir.
- raise_for_status 4xx/5xx yanıtlarını exception haline getirir.
- Retry yalnızca idempotent veya güvenli yeniden deneme politikasına uygun operasyonlarda kullanılmalıdır.
- API tokenları çevre değişkeni veya secret manager üzerinden alınmalıdır.

Mini görev

Bir API sonucunu 60 saniyelik dosya cache ile saklayan fetch_with_cache fonksiyonu tasarlayın.

MODÜL 11

Pandas ile Veri İşleme

Pandas tablo biçimindeki veriyi DataFrame nesnesi ile işler. CSV/Excel okuma, filtreleme, groupby, merge ve zaman serisi dönüşümleri raporlama işlerinde oldukça kullanışlıdır. Büyük veride veri tipi ve kopyalama maliyetleri izlenmelidir.

PANDAS PIPELINE

```
import pandas as pd

df = pd.read_csv("sales.csv")
summary = (
    df.assign(revenue=df["qty"] * df["unit_price"])
    .groupby("region", as_index=False)["revenue"].sum()
    .sort_values("revenue", ascending=False)
)
summary.to_csv("region_summary.csv", index=False)
```

İşlem	Pandas aracı
Filtre	df.loc[...]
Gruplama	groupby + agg
Birleştirme	merge / join
Eksik veri	isna, fillna, dropna

Veri kalitesi

Analize başlamadan önce sütun tipleri, boş değer oranı, tekillik ve beklenen aralık kontrollerini yapın.

MODÜL 12

CLI Otomasyonu, argparse ve Zamanlama

Tekrarlanan işleri script haline getirirken sabit değerleri kod içine gömmek yerine komut satırı argümanı veya yapılandırma kullanmak gerekir. argparse yerleşik CLI parser sağlar; çıkış kodları otomasyon zincirleri için önemlidir.

CLI ISKELETİ

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("input")
parser.add_argument("--format", choices=["json", "csv"], default="json")
args = parser.parse_args()

print(f"Processing {args.input} as {args.format}")
```

- 0 çıkış kodu başarılı işlemi, sıfır dışı kod hata durumunu ifade eder.
- Cron, Task Scheduler veya CI pipeline scriptleri zamanlayabilir.
- Logging çıktısını dosyaya ve konsola farklı handlerlarla yönlendirebilirsiniz.
- Idempotent otomasyon aynı girdiye yeniden çalıştığında istenmeyen yan etki üretmemelidir.

Proje düzeni

Script büyüdüğünde src/, tests/ ve pyproject.toml yapısına geçin; CLI, I/O ve saf iş kurallarını ayrı modüllerde tutun.

Mini görev

input.csv dosyasını alıp --min-price parametresine göre filtreleyen ve output.json yazan CLI aracı oluşturun.

MODÜL 13 - FINAL

Bitirme Projesi: Otomatik Operasyon Raporu

Proje senaryosu: Bir klasörde günlük CSV dosyaları ve bir HTTP API üzerinden durum verisi bulunmaktadır. Uygulama bu kaynakları birleştirip doğrulamalı, özet üretmeli ve JSON + CSV raporları oluşturmalıdır.

Zorunlu gereksinimler

- pathlib ile input klasöründeki CSV dosyalarını bul.
- Pandas ile birleştir; tarih ve sayısal sütunları doğru.
- HTTP API çağrısına timeout ve hata yönetimi ekle.
- En az bir generator ile büyük dosya/akış işle.
- argparse ile --date ve --output-dir argümanlarını destekle.
- logging ile INFO/WARNING/ERROR seviyeleri üret.
- pytest ile en az 6 test yaz; API çağrısını testte sahtele.

Kısa değerlendirme

Soru	Beklenen odak
1. list ile generator farkı?	Eager vs lazy üretim / bellek
2. set ne zaman avantajlı?	Tekillik ve üyelik testi
3. with ne sağlar?	Kaynak yaşam döngüsü
4. type hints ne kazandırır?	Statik analiz ve sözleşme
5. timeout neden zorunlu?	Kontrolsüz beklemeyi önleme

Başarı ölçütü

Aynı komut farklı gün girdileriyle tekrar çalışmalı, hata durumunda anlaşılır log üretmeli ve raporları deterministik biçimde oluşturmalıdır.